



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR 4

Planificación de caminos I

Algoritmo de Dijkstra sobre grafo topológico

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: ROS 2 Humble · CoppeliaSim · Pioneer 3DX

Índice

1. Introducción	3
1.1. Objetivo de la práctica	3
1.2. Descripción del problema	3
2. Fundamentos teóricos	3
2.1. Algoritmo de Dijkstra	3
2.2. Propiedades	4
2.3. Corte temprano	4
3. Diseño e implementación	4
3.1. Interfaz de la función	4
3.2. Validaciones de entrada	5
3.3. Bucle principal	5
3.4. Reconstrucción de la ruta	5
4. Resultados experimentales	6
4.1. Bloque A: grafo de ejemplo del enunciado (7 nodos)	6
4.2. Bloque B: grafos.mat (matriz J, 25 nodos, dirigida y asimétrica) . .	7
4.3. Casos límite verificados	8
5. Discusión	8
5.1. Decisiones de implementación	8
5.2. Limitaciones	9
6. Conclusiones	9

1. Introducción

1.1. Objetivo de la práctica

El objetivo es implementar el **algoritmo de Dijkstra** para calcular la ruta de coste mínimo entre dos nodos de un grafo topológico ponderado, partiendo de las matrices de adyacencia proporcionadas en el fichero `grafos.mat`. El resultado proporciona, para una pareja origen–destino, el coste total acumulado y la secuencia de nodos del camino óptimo.

1.2. Descripción del problema

Dado un grafo dirigido $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ con N vértices y una matriz de costes $G \in \mathbb{R}^{N \times N}$ tal que $G(i, j) > 0$ representa la existencia de un arco $i \rightarrow j$ con peso $G(i, j)$, y dos nodos s (origen) y t (destino), se desea encontrar:

1. el coste mínimo $d^*(s, t) = \sum_{(u,v) \in \pi^*} G(u, v)$,
2. la secuencia de nodos $\pi^* = [s, \dots, t]$ que lo realiza.

El convenio del enunciado es importante: los pares (i, j) con $G(i, j) = 0$ representan ausencia de arco (no autolazos de coste cero), y todos los pesos son no negativos, condición necesaria para la corrección del algoritmo.

2. Fundamentos teóricos

2.1. Algoritmo de Dijkstra

Dijkstra resuelve el problema del camino más corto desde un origen único en grafos con pesos no negativos. Mantiene tres estructuras:

- un vector de **distancias** $d[v]$ con la mejor estimación actual desde s hasta v ,
- un vector de **predecesores** $\pi[v]$ que permite reconstruir la ruta,
- un conjunto de nodos **visitados** (cerrados).

Inicialmente $d[s] = 0$, $d[v] = \infty$ para $v \neq s$, y todos los nodos están abiertos. El bucle principal selecciona en cada paso el nodo abierto u con menor $d[u]$, lo marca como visitado y **relaja** sus aristas salientes:

$$\text{si } d[u] + G(u, v) < d[v] \implies d[v] \leftarrow d[u] + G(u, v), \pi[v] \leftarrow u \quad (1)$$

El algoritmo termina cuando se extrae el nodo destino o cuando el menor d entre los abiertos es infinito (resto inalcanzable).

2.2. Propiedades

- **Optimalidad:** con pesos no negativos, una vez se extrae un nodo u del conjunto abierto, $d[u]$ es definitivo.
- **Complejidad:** $O(N^2)$ con la implementación por arrays empleada (selección lineal del mínimo). Una cola de prioridad reduciría a $O((N + E) \log N)$, pero para los tamaños de `grafos.mat` no es necesario.
- **Reconstrucción del camino:** a partir de π , retrocediendo desde t hasta s .

2.3. Corte temprano

Cuando solo interesa el camino a un destino concreto, se puede detener el bucle al extraer $u = t$. Esta optimización no cambia la complejidad asintótica pero reduce el trabajo medio.

3. Diseño e implementación

3.1. Interfaz de la función

La función se ha implementado en MATLAB con la signatura siguiente:

```
[coste, ruta] = dijkstra(G, origen, destino)
```

G puede pasarse como matriz $N \times N$ o como el struct devuelto por `load('grafos.mat')`. En este último caso, se extrae automáticamente el campo `J` del struct (matriz de adyacencia ponderada del enunciado) o, si no existe, la primera matriz cuadrada numérica que se localice. Esto permite invocar la función con `dijkstra(load('grafos.mat'), 1, 7)` sin pasos intermedios.

3.2. Validaciones de entrada

Antes del bucle principal se realizan las siguientes comprobaciones:

1. G cuadrada $N \times N$.
2. Costes finitos y no negativos (Dijkstra no soporta pesos negativos).
3. Índices origen y destino dentro de rango.
4. Si `origen == destino`, se devuelve `coste = 0` y `ruta = origen` sin entrar al bucle.
5. Aviso si la diagonal contiene autolazos no nulos (se ignoran al recorrer vecinos por el filtro $G(u, v) > 0$).

3.3. Bucle principal

El bucle implementa fielmente el esquema clásico:

1. Selecciona el nodo u no visitado con menor $d[u]$.
2. Si $d[u] = \infty$, los nodos restantes son inalcanzables: termina.
3. Marca u como visitado.
4. Si $u = t$, aplica corte temprano y termina.
5. Recorre los vecinos v con $G(u, v) > 0$ y aplica la relajación (1).

3.4. Reconstrucción de la ruta

Al finalizar, si $d[t] = \infty$ se devuelve `coste = Inf` y `ruta = []`. En caso contrario se reconstruye `ruta` retrocediendo por el vector de predecesores hasta el origen. Como salvaguarda, si en algún paso de la reconstrucción el predecesor es 0 (situación teóricamente imposible al haber llegado al destino con coste finito) se devuelve `coste = Inf` y `ruta = []`.

4. Resultados experimentales

4.1. Bloque A: grafo de ejemplo del enunciado (7 nodos)

El enunciado plantea como primer caso de validación el siguiente grafo de 7 nodos:

$$G = \begin{bmatrix} 0 & 2 & 0 & 0 & 9 & 0 & 0 \\ 2 & 0 & 1 & 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 5 & 0 & 0 & 0 \\ 0 & 2 & 5 & 0 & 1 & 2 & 4 \\ 9 & 0 & 0 & 1 & 0 & 4 & 0 \\ 0 & 0 & 0 & 2 & 4 & 0 & 1 \\ 0 & 0 & 0 & 4 & 0 & 1 & 0 \end{bmatrix}$$

La ejecución $[\text{coste}, \text{ruta}] = \text{dijkstra}(G, 1, 7)$ devuelve el resultado esperado: $\text{coste} = 7, \text{ruta} = [1 \ 2 \ 4 \ 6 \ 7]$ (Figura 1).

`dijkstra(G, 1, 7)` — coste = 7, ruta = [1 2 4 6 7]

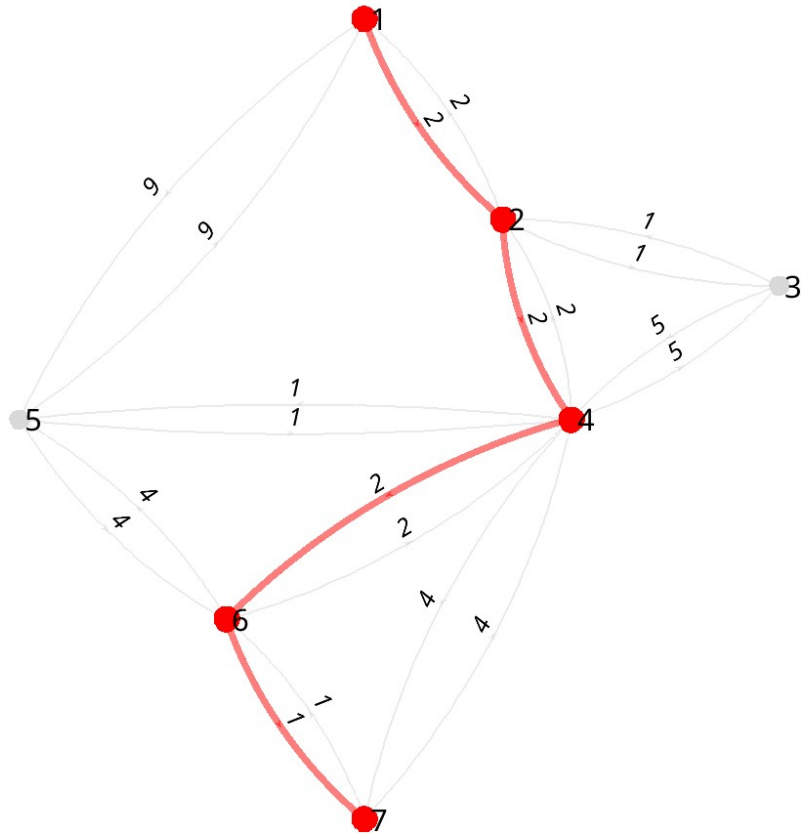


Figura 1: Grafo de 7 nodos del enunciado con la ruta óptima del nodo 1 al nodo 7 resaltada en rojo. Coste total 7, ruta [1 2 4 6 7].

4.2. Bloque B: grafos.mat (matriz J, 25 nodos, dirigida y asimétrica)

El segundo bloque de validación usa la matriz J contenida en `grafos.mat`, un grafo **dirigido y asimétrico** de 25 nodos. La asimetría es importante: por ejemplo el coste $1 \rightarrow 19$ (16) no coincide con $19 \rightarrow 1$ (19) precisamente porque $J(i, j) \neq J(j, i)$ en algunos arcos.

La Tabla 1 resume los 8 casos del enunciado y compara los resultados obtenidos por la implementación con los esperados. Todos coinciden.

Tabla 1: Validación de `dijkstra` sobre la matriz `J` de `grafos.mat` (25 nodos, dirigida).

Origen	Destino	Coste	Ruta	Estado
1	19	16	[1 7 13 17 23 18 19]	✓
19	1	19	[19 20 15 10 5 9 8 3 2 1]	✓
25	19	11	[25 20 15 14 18 19]	✓
19	25	4	[19 20 25]	✓
1	23	12	[1 7 13 17 23]	✓
23	1	22	[23 18 14 15 10 5 9 8 3 2 1]	✓
1	24	∞	[] (no alcanzable)	✓
23	22	40	[23 18 14 15 10 5 9 8 3 2 1 7 13 17 12 11 16 22]	✓

Obsérvese que las rutas de ida y vuelta entre el mismo par de nodos suelen **no ser simétricas**: por ejemplo $1 \rightarrow 23$ pasa por [1, 7, 13, 17, 23] con coste 12, mientras que $23 \rightarrow 1$ recorre [23, 18, 14, 15, 10, 5, 9, 8, 3, 2, 1] con coste 22. El caso $23 \rightarrow 22$ es paradigmático: aunque 22 y 23 son nodos próximos en el mapa, la única forma de alcanzar 22 desde 23 pasa por casi todo el grafo (coste 40, 18 nodos), porque los arcos disponibles no permiten un atajo directo.

4.3. Casos límite verificados

- `origen == destino`: devuelve coste 0 y ruta de un solo nodo.
- Nodos sin camino (caso $1 \rightarrow 24$): devuelve `coste = Inf`, `ruta = []`.
- Pesos negativos: rechazados por las validaciones de entrada con error explícito.

5. Discusión

5.1. Decisiones de implementación

Selección del mínimo por búsqueda lineal. Para los tamaños de los grafos del enunciado, una búsqueda lineal sobre los nodos abiertos es más simple y rápida en la práctica que mantener una cola de prioridad. La complejidad $O(N^2)$ es perfectamente aceptable.

Uso de `struct` de `load`. Aceptar directamente la salida de `load('grafos.mat')` reduce los pasos del usuario y evita errores al confundir cuál de las matrices del

fichero corresponde al grafo activo (se establece `J` como campo por defecto).

Convenio $G(i, j) = 0$ **como ausencia de arco.** Es el convenio del enunciado y se respeta al recorrer vecinos con `find(G(u,:) > 0)`. La advertencia de diagonal no nula previene errores al usar matrices preparadas para otros algoritmos.

5.2. Limitaciones

- No se soportan pesos negativos. Para esos casos habría que usar Bellman-Ford.
- No se devuelve la tabla completa de distancias a todos los nodos, solo al destino. Si fuera necesario para depurar, basta con eliminar el corte temprano y devolver `dist` y `prev`.
- La implementación no es óptima para grafos muy dispersos de gran tamaño; en ese caso, una cola de prioridad sería preferible.

6. Conclusiones

Se ha implementado una versión robusta y autocontenida del algoritmo de Dijkstra adaptada al formato de datos del enunciado. La interfaz acepta tanto matrices como el `struct` de `load`, lo que simplifica su uso interactivo en MATLAB. La función incorpora las salvaguardas habituales (validación de pesos, casos límite, corte temprano y manejo de inalcanzabilidad) y proporciona los dos elementos esperados: coste mínimo y secuencia de nodos del camino óptimo.

Esta función constituye la base de la planificación global utilizada después en Lab-MR 5 (variante A* con heurística consistente) y en Lab-MR 6 (navegación autónoma completa, donde la ruta de Dijkstra actúa como secuencia de subobjetivos para la navegación local reactiva).